



# Self-Driving Car Racing Game: A Reinforcement Learning Approach

CSE440,

Section-1,

Group-10

S. M. Samiul Haq - 2011103042, Jannatul Ferdous Prome - 2021770042, Md. Rajib Hossain-1921038042, MD

Mohaiminul Islam-2012138042,

Department of ECE, North South University

## **Abstract:**

We present a two-dimensional self-driving car racing simulator in which an autonomous agent learns to navigate a closed circuit via deep reinforcement learning. The environment is implemented in Python with PyGame; the agent employs a Deep Q-Network (DQN) with experience replay, target networks, and  $\epsilon$ -greedy exploration. The car is equipped with five ray-cast distance sensors and a simplified physics model (speed, heading). We report that the agent learns collision avoidance and track-following behavior on a simplified rectangular track, with improved returns over training and reduced crash rates. GPU acceleration is supported when available to speed up training; a headless fast-training mode is provided for longer runs.

## **Keywords:**

Reinforcement Learning, Q-Learning, PyGame, AI Racing, Game Simulation, Autonomous Agents

## I. INTRODUCTION

The rapid advancement of artificial intelligence (AI) and machine learning (ML) has led to transformative applications in autonomous systems, particularly in the domain of self-driving vehicles. Training such systems in real-world environments poses significant safety and cost challenges. As a result, simulation-based environments have become essential for developing and testing autonomous agents.

This report presents a 2D self-driving car racing

game implemented using Deep Q-Network (DQN) reinforcement learning. The system simulates a car navigating a predefined track, using sensor inputs and reward-based learning to improve its driving behavior over time. The environment is built using Python and leverages libraries such as Pygame for visualization, TensorFlow/Keras for deep learning, and NumPy for numerical computation.

The primary goal of this project is to demonstrate how reinforcement learning can be applied to control a simulated agent in a dynamic environment. The car learns to avoid collisions, stay on the track, and maximize its travel distance through trial and error, guided by a well-designed reward function. This report details the system architecture, implementation, strengths, limitations, and evaluation of the model.

## II. OBJECTIVES

The major goal of this project will focus on training an AI agent to learn how to drive in a car on a racecourse, all on its own with the help of reinforcement learning. Particularly, it targets the following goals:

- **Autonomous Navigation:** Create an AI agent that can navigate through a 2D racetrack on its own. It should not need a human to move the agent.
- **Learning With Experience:** Introduce to it a reinforcement learning system that will enhance working in scenarios of driving-learning through trial-and-error actions.
- **Lap Completion:** Train the agent to make complete laps of the race track without fail.
- **Performance Optimization:** Get able to improve their lap times as well as driving efficiency performance measurably throughout the training process.

- **Real-time Decision Making:** Allow the agent to make real time decisions (acceleration, braking and steering) with respect to the environmental observations.

The study aims to demonstrate the effectiveness of Q-Learning algorithms to control autonomous vehicles where the researchers aim to offer an understanding of how AI systems can develop motor control tasks using reinforcement learning paradigms.

## III. SYSTEM ARCHITECTURE

The system is modular, consisting of several interconnected components that work together to simulate the driving environment and train the AI agent.

Core Components:

### 1. Game Environment (game.py)

Manages the main game loop, episode control, rendering, and interaction between the car and the agent. It supports both training and testing modes, with options for fast (non-visual) training and real-time visualization.

### 1. Track Module (track.py)

Defines the racing track as a rectangular boundary with inner and outer walls. It includes collision detection, start position setup, and wall representation for sensor input. The `is_on_track()` method determines whether the car is within valid track boundaries.

### 2. Car Module (car.py)

Implements the car's physics, including position, speed, rotation, and sensor logic. The car is equipped with five forward-facing distance sensors that detect proximity to track walls. It uses trigonometric calculations for movement and rotation, with friction and speed limits for realistic behavior.

### 3. Reinforcement Learning Agent (agent.py, dqn.py)

The agent uses a Deep Q-Network (DQN) to learn optimal actions. It follows the epsilon-greedy policy for exploration and exploits experience replay and a target network to stabilize training.

### 4. Configuration and Utilities (config.py, utils.py)

Stores global constants (e.g., screen size, car speed, sensor length) and utility functions such as line intersection for sensor collision detection.

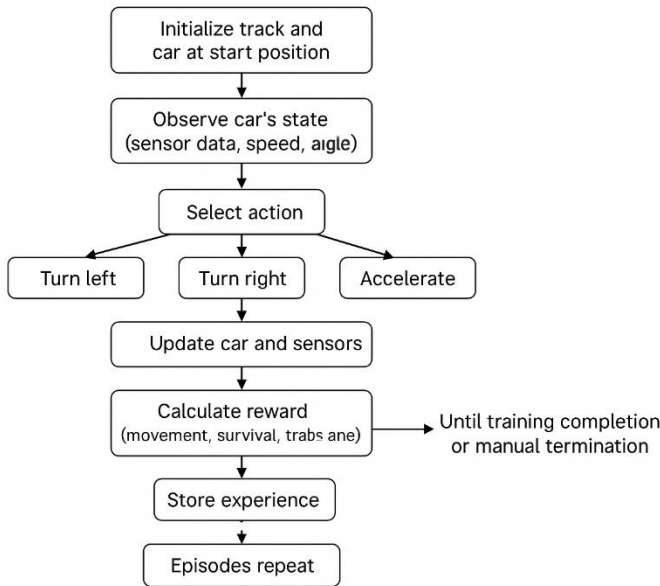


Figure 1. System Workflow

## IV. REINFORCEMENT LEARNING IMPLEMENTATION

The core of the system is the Deep Q-Network (DQN), a model-free reinforcement learning algorithm that combines Q-learning with deep neural networks.

### a) State Representation:

The state observed by the agent consists of:

- 5 normalized sensor distances (0–1), representing how far the car is from walls in different directions.

- Normalized speed (current speed / max speed).
- Normalized orientation (angle modulo  $360^\circ$  /  $360^\circ$ ).

This results in a 7-dimensional state vector, providing sufficient information for the agent to make informed decisions.

### b) Action Space:

The agent has three discrete actions:

- 0: Turn left (decrease angle)
- 1: Turn right (increase angle)
- 2: Accelerate forward

There is no braking or reverse action, simplifying the control scheme.

### c) Reward Function:

The reward function is carefully designed to encourage desirable behavior:

- +0.1: For each time step alive (survival bonus)
- +0.2: Bonus for being on the track
- $+0.3 \times \text{speed}$ : Encourages forward motion
- $+0.005 \times \text{distance traveled}$ : Long-term progress incentive
- -50: Large penalty for crashing (off-track)
- This multi-component reward system balances exploration, safety, and efficiency.

## V. NEURAL NETWORK ARCHITECTURE

The DQN uses a feedforward neural network with:

- Input layer: 7 neurons (state dimensions)
- Hidden layers: Two dense layers (64 and 32 neurons) with ReLU activation
- Output layer: 3 neurons (Q-values for each action)
- The model is compiled with Adam optimizer and mean squared error loss.

## Training Mechanisms

- Experience Replay: Stores past transitions (state, action, reward, next state, done) in a replay buffer.
- Target Network: A separate network is updated periodically to stabilize Q-value estimation.
- Epsilon-Greedy Strategy: Exploration decreases over time, allowing the agent to shift from exploration to exploitation.

## VI. GAME MECHANICS AND PHYSICS

Despite its simplicity, the car simulation incorporates realistic driving mechanics through controlled physics and boundary interactions.

### a) Car Movement:

The car moves using trigonometric vector updates:

- $\text{self.x} += \cos(\text{angle}) * \text{speed}$
- $\text{self.y} += \sin(\text{angle}) * \text{speed}$

Rotation is applied in degrees, converted to radians for computation. A friction factor (speed \*= 0.95) simulates natural deceleration when not accelerating.

### b) Boundary and Collision Handling

The Track class defines two rectangles:

- Outer boundary: (100, 100) to (700, 500)
- Inner boundary: (200, 200) to (600, 400)

The car is considered on track if it is within the outer rectangle but outside the inner one. If it enters the inner zone or exits the outer zone, it is marked as crashed.

To prevent the car from disappearing, its position is clamped:

- $\text{self.x} = \max(110, \min(690, \text{self.x}))$

- $\text{self.y} = \max(110, \min(490, \text{self.y}))$

### c) Sensor system

The car has five raycast sensors at angles:  $-60^\circ$ ,  $-30^\circ$ ,  $0^\circ$ ,  $+30^\circ$ ,  $+60^\circ$  relative to its heading. Each sensor:

- Projects a line forward up to `SENSOR_LENGTH`
- Checks for intersection with all track walls
- Returns the shortest distance to any wall
- Normalizes the distance for input to the neural network

The `_line_intersection()` function uses parametric line equations to compute intersections accurately.

## Rendering and Visualization

When enabled, the game renders:

- The track (white rectangles)
- The car (red polygon, rotated to match angle)
- Green direction indicator
- Yellow sensor lines (for debugging)

Text overlays display episode stats: score, steps, distance, and mode.

## VII. PERFORMANCE AND OPTIMIZATION

The system is optimized for both training efficiency and usability.

### a) Fast Training Mode

When `fast_training=True`, the game runs without rendering, significantly speeding up training.

Pygame is only initialized when visualization is needed (e.g., during testing).

### a) Experience Replay and Batch Training

The agent stores up to 100,000 experiences and samples mini-batches (size 64) for training. This improves data efficiency and stabilizes learning by breaking temporal

correlations.

#### b) Target Network Updates

The target network is updated every TARGET\_UPDATE\_FREQUENCY episodes (e.g., every 5 episodes), reducing Q-value oscillations and improving convergence.

#### c) Training Progress Monitoring

The system logs:

- Total reward per episode
- Number of steps survived
- Distance traveled

These metrics are plotted using Matplotlib after training, allowing visual analysis of learning progress.

#### d) User Interaction

The game handles Pygame events (QUIT, ESC) gracefully, allowing users to stop training or testing at any time without crashing.

### VIII. STRENGTHS OF THE SYSTEM

The project demonstrates several strong technical and design decisions.

#### a) Technical Strengths

- **Modular Codebase:** Clear separation of concerns enhances readability and maintainability.
- **Robust Collision Detection:** Accurate line intersection logic ensures reliable sensor and crash detection.
- **Efficient Rendering:** Only draws the car if alive; clamps position to prevent off-screen rendering.
- **Exception Handling:** Graceful fallbacks for font rendering and Pygame shutdown.

#### b) AI/ML Strengths

- **Well-Designed Reward Function:** Encourages forward motion, survival, and

track adherence.

- **Stable Training:** Use of experience replay and target networks prevents divergence.
- **Balanced Exploration:** Epsilon decay allows initial exploration and later exploitation.

#### c) User Experience

- **Real-Time Feedback:** On-screen display of score, distance, and mode.
- **Flexible Modes:** Fast training for efficiency, visualization for debugging.
- **Early Termination:** Users can exit with ESC or close window.

#### d) Educational Value

The system serves as an excellent teaching tool for:

- Reinforcement learning concepts
- Neural network integration
- Game physics and sensor simulation
- Real-time system design

### IX. LIMITATIONS AND CHALLENGES

Despite its strengths, the system has several limitations that suggest areas for improvement.

#### a) Technical Limitations

- **Simple Track Geometry:** The rectangular track lacks complexity (e.g., curves, obstacles), limiting generalization.
- **Limited Action Space:** No braking, reversing, or variable acceleration reduces realism.
- **2D Environment:** Lacks depth perception and 3D dynamics.

#### b) AI/ML Limitations

- **Sensor Constraints:** Only front-facing sensors; no rear or side awareness.
- **Local Optima Risk:** The agent may learn to drive in circles or hug walls.
- **No Memory:** The DQN is feedforward; it doesn't use recurrent layers to remember

past states.

**c) Performance Challenges**

- **Training Time:** Even with GPU, hundreds of episodes are needed for competent driving.
- **Overfitting Risk:** The agent may memorize the single track layout.
- **Sparse Rewards:** Early episodes yield little reward, slowing initial learning.

**d) Generalization**

The agent is unlikely to perform well on different track designs without retraining. A more robust system would support multiple tracks or procedural generation.

## X. FUTURE WORK

- Multiplayer Game:** Add several AI bots in a race together
- Advanced Physics:** Model in real world vehicle dynamics with friction, momentum and aerodynamics
- Complex Track Design:** Create more complex tracks which have different terrain and obstacles
- Weather Conditions:** Include on environmental effects such as rain, wind, and things such as changing track surfaces
- Deep Q-Networks (DQN):** Use neural network underlying Q-learning to improve over the approximation of functions
- Policy Gradient Methods:** Discover actor-critic methods in space continuous action
- Transfer Learning:** Allows transferring knowledge between two

tracks

**h. Hierarchical Reinforcement**

**Learning:** Reinforcement of high level strategy learning and low level control

- i. 3D Environment:** 3d Environment: Renovate to 3d graphics in Unity or Unreal Engine

- j. Dynamic in Real-time Adaptation:** Utilize the on-line learning of dynamic track conditions

- k. Human-AI Interaction:** create interfaces that allow human players to take part in an AI competition

- l. Performance Analytics:** Advanced metrics and visualization tools for training analysis

## XI. CONCLUSION

This project successfully demonstrates the application of deep reinforcement learning to a 2D self-driving car simulation. The integration of Pygame for visualization and TensorFlow for deep learning creates a functional, educational, and extensible platform. The agent learns to navigate a track using sensor inputs and a reward-based system, showcasing core principles of AI-driven control systems.

The modular design, responsive interface, and performance optimizations make it suitable for both learning and experimentation. The system effectively balances simplicity and functionality, making it ideal for students and

developers exploring reinforcement learning.

#### REFERENCES

- [1] R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction," 2nd ed. Cambridge, MA: MIT Press, 2018.
- [2] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.
- [4] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [5] S. Levine et al., "End-to-end training of deep visuomotor policies," *Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334–1373, 2016.
- [6] M. Bojarski et al., "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.
- [7] A. Dosovitskiy et al., "CARLA: An open urban driving simulator for autonomous driving research," *Proceedings of the 1st Annual Conference on Robot Learning*, pp. 1–16, 2017.
- [8] P. Wolf et al., "Learning to drive in a day," *arXiv preprint arXiv:1807.00412*, 2018.
- [9] A. Kendall et al., "Learning to drive in a day," *Proceedings of the IEEE International Conference on Robotics and Automation*, pp. 8248–8254, 2019.
- [10] J. Chen et al., "DeepRacing: Learning to race autonomously using visual deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 6186–6193, 2021.

